

CS121 Data Structures

Quadratic Sorting

Apig Aramian
aaramian@aua.am



Fall 2026

Important Data and Statistics



- ▶ 26 classes remaining
- ▶ 39 days till the Midterm exam I

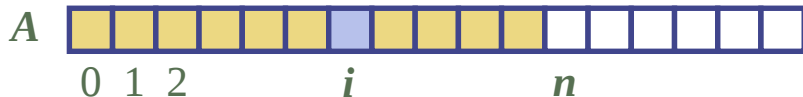
Array Definition

An **array** is a sequenced collection of variables all of the same type.

Each variable, or **cell**, in an array has an **index**, which uniquely refers to the value stored in that cell.

The cells of an array A , are numbered 0, 1, 2, and so on.

Each value stored in an array is often called an array **element**.



Array Length and Capacity

The length of an array determines the maximum number of things that can be stored in the array

We will sometimes refer to the length of an array as its **capacity**.

In Java, the length of an array named *a* can be accessed using the syntax `a.length`.

Thus, the cells of an array *a* are numbered 0, 1, 2, and so on, up through `a.length - 1`.

The cell with index *k* can be accessed with syntax `a[k]`.

Sorting an Array

Sorting problem: start with an unordered array of elements and rearrange them into nondecreasing order

- ▶ Selection Sort

- ▶ Insertion Sort

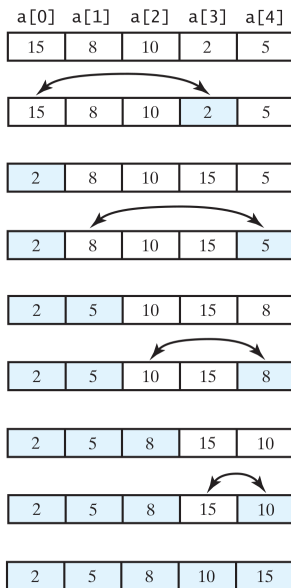
- ▶ Bubble Sort

Selection Sort

The **selection sort** algorithm works as follows:

1. Find (**select**) the smallest element in the array, and exchange it with the element in the first position
2. Find the second smallest element and exchange it with the element in the second position
3. Continue in this way until the entire array is sorted

Selection Sort: Illustration



Selection Sort: An Implementation

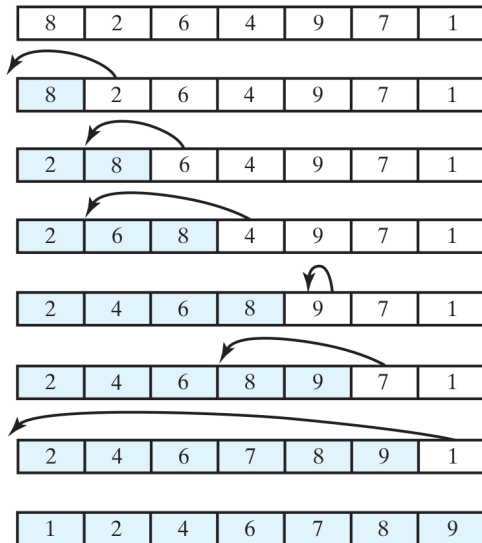
```
1  /** Selection-sort of an array of integers into nondecreasing order */
2  public static void selectionSort(int[ ] data) {
3      int n = data.length;
4      for (int k = 0; k < n - 1; k++) {          // begin with first position
5          int min = k;                          // position of current minimum
6          for (int j = k + 1; j < n; j++)        // start at next position
7              if (data[j] < data[min])          // a new minimum found
8                  min = j;                     // update position of minimum
9          int temp = data[k];                   // put minimum in correct position
10         data[k] = data[min];
11         data[min] = temp;
12     }
13 }
```

Insertion Sort

The **insertion sort** algorithm proceeds by considering one element at a time, placing (**inserting**) the element in the correct position relative to those before it.

1. First element—trivially sorted by itself
2. Second element: if it is smaller than the first, swap them
3. Third element: swap it leftward until in its proper place
4. Continue in this way until the entire array is sorted

Insertion Sort: Illustration



Insertion Sort: An Implementation

```
1  /** Insertion-sort of an array of integers into nondecreasing order */
2  public static void insertionSort(int[ ] data) {
3      int n = data.length;
4      for (int k = 1; k < n; k++) {           // begin with second integer
5          int cur = data[k];                 // time to insert cur=data[k]
6          int j = k;                         // find correct index j for cur
7          while (j > 0 && data[j-1] > cur) {  // thus, data[j-1] must go after cur
8              data[j] = data[j-1];          // slide data[j-1] rightward
9              j--;                           // and consider previous j for cur
10         }
11         data[j] = cur;                     // this is the proper place for cur
12     }
13 }
```

Bubble Sort

Keep passing through the array, swapping adjacent elements that are out of order, continuing until the array is sorted.

Assume at each outer pass, we '**bubble**' from the beginning of the array to its end.

After one pass, the largest element is in place.

After a second pass, the second largest element is in place.

And so on, until all elements are in place.

Bubble Sort: Illustration

Original array

8	2	6	4	9	7	1
---	---	---	---	---	---	---

After pass 1

						Sorted
2	6	4	8	7	1	9

After pass 2

					Sorted	
2	4	6	7	1	8	9

After pass 3

				Sorted		
2	4	6	1	7	8	9

After pass 4

			Sorted			
2	4	1	6	7	8	9

After pass 5

		Sorted				
2	1	4	6	7	8	9

After pass 6

Sorted						
1	2	4	6	7	8	9

Bubble Sort: An Implementation

```
1  /** Bubble-sort of an array of integers into nondecreasing order */
2  public static void bubbleSort(int[ ] data) {
3      int n = data.length;
4      for (int k = 0; k < n - 1; k++) {          // find last k + 1 elements
5          boolean swapped = false;
6          for (int j = 0; j < n - k - 1; j++) // consider successive elements
7              if (data[j] > data[j + 1]) {      // compare successive elements
8                  int temp = data[j];           // swap successive elements
9                  data[j] = data[j + 1];
10                 data[j + 1] = temp;
11                 swapped = true;
12             }
13         if (!swapped)                          // stop if there were no swaps
14             break;
15     }
16 }
```

Sorting in Quadratic Time

Sorting problem: start with an unordered array of elements and rearrange them into nondecreasing order

- ▶ Selection Sort
- ▶ Insertion Sort
- ▶ Bubble Sort

All three algorithms have quadratic complexity, i.e. $O(n^2)$.

Summary

Reading

Section 3.1 Using Arrays

Questions?